

وظيفة البرمجة التفرعية

يهدف المشروع إلى بناء منصة تجارة إلكترونية متطورة تركز على حل تحديات التزامن "Concurrency".

حيث لا يقتصر المشروع على البيع والشراء التقليدي بل يتمحور حول ضمان استقرار النظام وسلامة البيانات عند تعرضه لضغط كبير من الطلبات المتزامنة في وقت واحد.

يركز المشروع على تحقيق التوازن بين كفاءة الأداء ودقة البيانات حيث تكمن أهدافنا الأساسية في منع تضارب العمليات (Race Conditions) وتنظيم استهلاك موارد الخادم (CPU & RAM) وضمان استجابة سريعة للمستخدم حتى في أصعب ظروف التحميل المتزامن.

تتمثل الوظائف الأساسية للنظام -والتي تمثل المتطلبات الوظيفية- في إدارة عملية الشراء بدقة وتشمل:

- إدارة المخزون: تحديث كميات المنتجات في قاعدة البيانات فور إتمام كل عملية.
- معالجة الطلبات: استقبال طلبات الشراء والتحقق من توفر الكمية المطلوبة قبل الخصم.
- سجل العمليات: إنشاء سجلات دقيقة لكل طلب شراء ناجح لضمان تتبع البيانات.

هيكل قاعدة البيانات (Database Schema):

جدول المنتجات (Products): وهو الجدول المحوري الذي يحتوي على بيانات المنتجات مع حقل "المخزون (Stock)" الذي نطبق عليه عمليات القفل (Locking) لضمان دقة الكميات.

	id	name	price	stock	created_at	updated_at
☐ Edit ☐ Copy ☐ Delete	1	Laptop AI Edition	1200.00	1	2026-05-09 12:43:46	2026-05-15 12:08:55

جدول الطلبات (Orders): يُستخدم لتوثيق العمليات الناجحة حيث يربط كل عملية شراء بالمنتج الخاص بها ويسجل الكمية التي تم بيعها.

				id	user_id	total_price	status	created_at	updated_at
☐	Edit	Copy	Delete	1	1	4800.00	completed	2026-05-10 16:44:43	2026-05-10 16:44:43
☐	Edit	Copy	Delete	2	1	4800.00	completed	2026-05-10 16:44:43	2026-05-10 16:44:43
☐	Edit	Copy	Delete	3	1	1200.00	completed	2026-05-10 17:01:50	2026-05-10 17:01:50
☐	Edit	Copy	Delete	4	1	1200.00	completed	2026-05-10 17:01:50	2026-05-10 17:01:50
☐	Edit	Copy	Delete	5	1	1200.00	completed	2026-05-10 17:11:18	2026-05-10 17:11:18
☐	Edit	Copy	Delete	6	1	1200.00	completed	2026-05-15 11:17:28	2026-05-15 11:17:28
☐	Edit	Copy	Delete	7	1	1200.00	completed	2026-05-15 11:44:26	2026-05-15 11:44:26
☐	Edit	Copy	Delete	8	1	1200.00	completed	2026-05-15 11:44:46	2026-05-15 11:44:46
☐	Edit	Copy	Delete	9	1	1200.00	completed	2026-05-15 11:44:54	2026-05-15 11:44:54
☐	Edit	Copy	Delete	10	1	1200.00	completed	2026-05-15 11:49:27	2026-05-15 11:49:27

جدول المهام (Jobs): وهو الجدول المسؤول عن دعم المعالجة غير المتزامنة حيث تُخزن فيه المهام الثقيلة (مثل تأكيد الطلب) مؤقتاً حتى يتم معالجتها في الخلفية.

				id	queue	payload	attempts	reserved_at	available_at	created_at
☐	Edit	Copy	Delete	8	default	("uuid":"9498e969-36b9-4bab-ad45-41ff90eb39f2","di...	1	1778846836	1778846836	1778846836

جدول تفاصيل الطلبات (Order Items): هو الجدول الذي يربط الطلب بالمنتج، حيث يسجل لكل طلب ما هي المنتجات التي تم شراؤها وبأي كمية وبأي سعر في تلك اللحظة.

				id	order_id	product_id	quantity	price	created_at	updated_at
☐	Edit	Copy	Delete	1	1	1	4	1200.00	2026-05-10 16:44:43	2026-05-10 16:44:43
☐	Edit	Copy	Delete	2	2	1	4	1200.00	2026-05-10 16:44:43	2026-05-10 16:44:43
☐	Edit	Copy	Delete	3	3	1	1	1200.00	2026-05-10 17:01:50	2026-05-10 17:01:50
☐	Edit	Copy	Delete	4	4	1	1	1200.00	2026-05-10 17:01:50	2026-05-10 17:01:50
☐	Edit	Copy	Delete	5	5	1	1	1200.00	2026-05-10 17:11:18	2026-05-10 17:11:18
☐	Edit	Copy	Delete	6	6	1	1	1200.00	2026-05-15 11:17:28	2026-05-15 11:17:28
☐	Edit	Copy	Delete	7	7	1	1	1200.00	2026-05-15 11:44:26	2026-05-15 11:44:26
☐	Edit	Copy	Delete	8	8	1	1	1200.00	2026-05-15 11:44:46	2026-05-15 11:44:46
☐	Edit	Copy	Delete	9	9	1	1	1200.00	2026-05-15 11:44:54	2026-05-15 11:44:54
☐	Edit	Copy	Delete	10	10	1	1	1200.00	2026-05-15 11:49:27	2026-05-15 11:49:27
☐	Edit	Copy	Delete	11	11	1	1	1200.00	2026-05-15 12:07:08	2026-05-15 12:07:08
☐	Edit	Copy	Delete	12	12	1	1	1200.00	2026-05-15 12:07:16	2026-05-15 12:07:16
☐	Edit	Copy	Delete	13	13	1	1	1200.00	2026-05-15 12:08:55	2026-05-15 12:08:55

معالجة المتطلبات الغير وظيفية:

1. الطلب الأول:

سلامة البيانات والتزامن (Data Integrity & Concurrency):

في أنظمة المتجر التقليدية، إذا كان هناك قطعة واحدة فقط من منتج ما وحاول شخصان الضغط على زر "شراء" في نفس الأجزاء من الثانية قد يقرأ النظام لكليهما أن الكمية متاحة فيقوم ببيع القطعة مرتين ويصبح المخزون بالسالب.

لحل هذه المشكلة استخدمنا تقنية Pessimistic Locking عبر دالة lockForUpdate()

```
$product = Product::where('id', $productId)->lockForUpdate()->first();
```

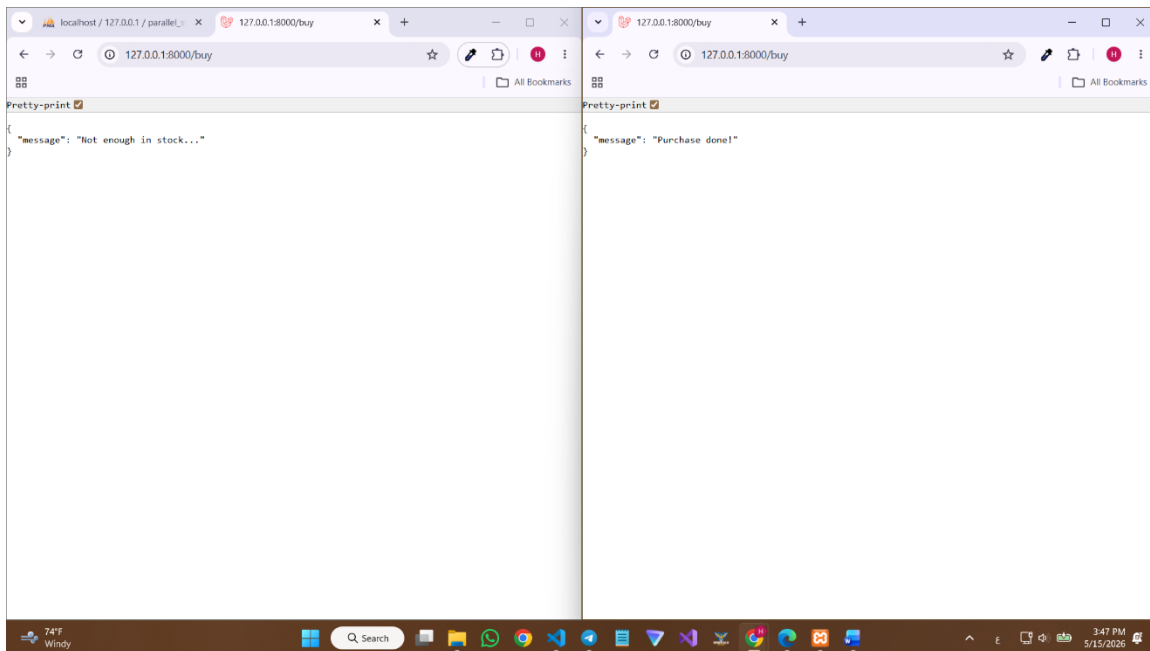
عندما يبدأ المستخدم الأول عملية الشراء يقوم السيرفر بوضع قفل (Lock) على سطر المنتج في قاعدة البيانات حيث إذا حاول المستخدم الثاني الدخول في نفس اللحظة سيجده مقفلاً فيضطر للانتظار حتى تنتهي العملية الأولى تماماً.

الاختبار:

- تجهيز بيئة الاختبار بضبط كمية المخزون لمنتج معين إلى (1) فقط لاختبار استجابة النظام عند محاولة سحب كميات تفوق المتاح تحت الضغط المتزامن.

	id	name	price	stock	created_at	updated_at
☐	1	Laptop AI Edition	1200.00	1	2026-05-09 12:43:46	2026-05-15 12:08:55

- بعد الضغط على زر الشراء في الشاشتين بشكل متوازي يظهر نجاح الطلب الأول أقصى اليمين وفشل الطلب الثاني أقصى اليسار برسالة 'الكمية غير كافية' مما يثبت فعالية الـ **Pessimistic Locking** في منع البيع الوهمي.



- يظهر تحديث المخزون إلى القيمة (0) بدقة مع تسجيل طلب واحد فقط في جدول العمليات، مما يؤكد عدم حدوث أي تضارب.

	id	name	price	stock	created_at	updated_at
	1	Laptop AI Edition	1200.00	0	2026-05-09 12:43:46	2026-05-15 12:47:19

	id	queue	payload	attempts	reserved_at	available_at	created_at
	13	default	{"uuid":"a8e895de-1031-4796-a96c-c2fb51d7406","di...	1	1778849537	1778849536	1778849536

2. الطلب الثاني:

إدارة السعة وحماية النظام (Capacity & Rate Limiting):

عندما يقوم مستخدم واحد (أو بوت) بإرسال مئات الطلبات في ثانية واحدة قد يتسبب ذلك في إغراق السيرفر وقاعدة البيانات مما يؤدي لبطء شديد أو توقف الخدمة عن بقية المستخدمين .

لحل هذه المشكلة استخدمنا Rate Limiter Middleware يقوم بعدّ الطلبات القادمة من كل مستخدم بناءً على عنوان IP الخاص به ، إذا تجاوز المستخدم الحد المسموح به يتم حظره مؤقتاً ومنع طلبه من الوصول للكود الأساسي أو قاعدة البيانات.
قمنا ببرمجة النظام ليسمح بـ 5 طلبات فقط في الدقيقة.

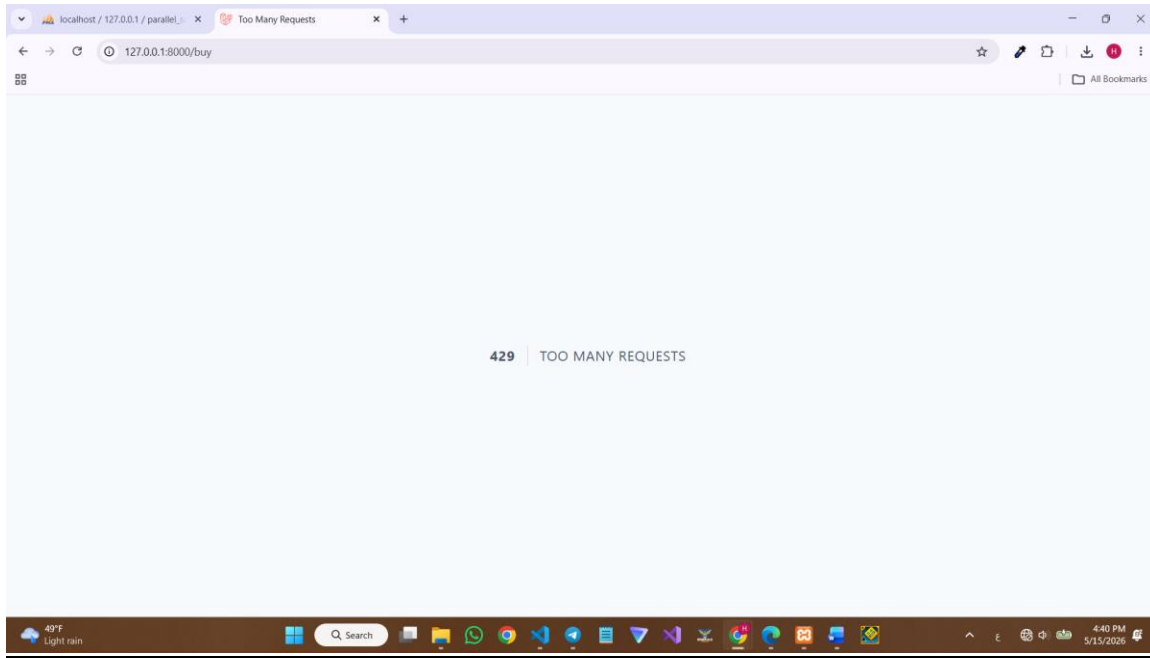
```
public function boot(): void  
  
RateLimiter::for('purchase_limit', function (Request $request) {  
    return Limit::perMinute(5)->by($request->ip());  
});
```

الاختبار:

- نرفع عدد المنتجات في الstock الى 6 حتى نختبر الضغط على زر الشراء 6 مرات

	id	name	price	stock	created_at	updated_at
☐ Edit ☐ Copy ☐ Delete	1	Laptop AI Edition	1200.00	6	2026-05-09 12:43:46	2026-05-15 13:38:29

- نضغط على زر الشراء 6 مرات فنجد ان النظام استجاب للضغوطات الـ 5 الأولى ولكن عند الضغطة السادسة تظهر شاشة الخطأ بأن هناك الكثير من الطلبات حدثت خلال هذه الدقيقة



- نجد أن العدد في الـ stock أصبح 1 بدل ان يصبح 0 على الرغم من الضغط 6 مرات

	id	name	price	stock	created_at	updated_at
☐ Edit ☐ Copy ☐ Delete	1	Laptop AI Edition	1200.00	1	2026-05-09 12:43:46	2026-05-15 13:40:41

2026-05-15 16:43:01 /buy	~ 501.33ms
2026-05-15 16:43:01 /buy	~ 0.15ms
2026-05-15 16:43:01 /buy	~ 4.41ms
2026-05-15 16:43:01 /buy	~ 501.08ms
2026-05-15 16:43:01 /buy	~ 496.29ms
2026-05-15 16:43:02 /buy	~ 1.06ms
2026-05-15 16:43:02 /favicon.ico	~ 1.06ms

3. الطلب الثالث:

الأداء والمعالجة المتوازية (Performance - Asynchronous Queues):

هناك مهام ثقيلة برمجياً مثل إرسال إيميل تأكيد أو تحديث سجلات إحصائية ضخمة أو محاكاة عملية معالجة تستغرق وقتاً.

إذا نفذنا هذه المهام مباشرة أثناء طلب الشراء سيضطر المستخدم للانتظار لثوانٍ طويلة أمام شاشة بيضاء قبل أن يعرف هل نجحت العملية أم لا مما يؤدي لتجربة مستخدم سيئة.

لحل هذه المشكلة استخدمنا نظام الطوابير "Queues" ف بدلاً من تنفيذ المهمة الثقيلة فوراً يقوم النظام بإنشاء تذكرة للمهمة ويضعها في جدول jobs في قاعدة البيانات ثم يُرسل النظام رداً فورياً للمستخدم بكلمة "تم النجاح".

يقوم محرك خلفي (Queue Worker) بمعالجة هذه المهام واحدة تلو الأخرى بعيداً عن عين المستخدم.

```
public function handle(): void
{
    sleep(5);
    \Log::info("Process done in the background");
}
```

الاختبار:

- نقوم بإيقاف المحرك عبر تعليمة php artisan queue:clear

```
C:\UNI Projects\parallel-store>php artisan queue:clear
INFO Cleared 11 jobs from the [default] queue.
```

- نضغط على زر الشراء ف ستظهر رسالة النجاح فوراً في أقل من ثانية رغم وجود تأخير 5 ثوانٍ في الكود.

```
{
    "message": "Purchase done!"
}
```

- سنجد سطوراً جديداً ظهر هناك، وهذا يعني أن المهمة منتظرة ولم يتم تنفيذها بعد.

	id	queue	payload	attempts	reserved_at	available_at	created_at
	25	default	["uuid":"ed9e6fca-b916-47db-b7a9-e8f94624b31f","di...	0	NULL	1778853552	1778853552

- نقوم بتفعيل محرك الطوابير عبر الأمر `php artisan queue:work` فنجد ان المهمة يتم تنفيذها الذي يستغرق 5 ثوان بسبب وجود `sleep`.

```
C:\UNI Projects\parallel-store>php artisan queue:work
2026-05-15 14:02:58 App\Jobs\ProcessOrderConfirmation ..... RUNNING
2026-05-15 14:03:03 App\Jobs\ProcessOrderConfirmation ..... 5s DONE
```

4. الطلب الرابع:

المعالجة المجمعّة المتوازية (Parallel Batch Processing)

ند التعامل مع كميات ضخمة من البيانات (مثل معالجة آلاف طلبات المبيعات اليومية لتوليد تقارير إحصائية)، فإن معالجتها بشكل فردي تستهلك وقتاً طويلاً وتستنزف موارد قاعدة البيانات. تم استخدام تقنية **Batching** في Laravel لتقسيم العمليات الكبيرة إلى دفعات (Chunks) ومعالجتها بالتوازي.

التنفيذ البرمجي :

قمنا بإنشاء Job يسمى `ProcessDailySalesBatch` يقوم بتقسيم البيانات إلى مجموعات صغيرة، واستخدام دالة `Bus::batch` لتنفيذ هذه المجموعات بشكل متوازٍ، مما يقلل الوقت الإجمالي للمعالجة بنسبة كبيرة.

```
16  {
17  }
18  //
19  /**
20   * Execute the job.
21   */
22  public function handle(): void
23  {
24  }
25  }
26
27  \App\Models\Order::whereDate('created_at', now()->today())
28  ->chunk(100, function ($orders) {
29      $batchJobs = [];
30
31      foreach ($orders as $order) {
32          $batchJobs[] = new \App\Jobs\ProcessOrderConfirmation($order);
33      }
34
35      if (count($batchJobs) > 0) {
36          \Illuminate\Support\Facades\Bus::batch($batchJobs)->dispatch();
37          \Illuminate\Support\Facades\Log::info("تم إطلاق دفعة معالجة متوازية اليوم - " . count($batchJobs) . " طلب.");
38      }
39  });
40
41  if (\App\Models\Order::whereDate('created_at', now()->today())->count() == 0) {
42      \Illuminate\Support\Facades\Log::warning("لم يتم العثور على طلبات جديدة لمعالجتها اليوم.");
43  }
44  }
```


الاختبار والنتائج: عند تشغيل الرابط المخصص للاختبار، يقوم النظام بإطلاق مجموعة من المهام في الخلفية. يمكن ملاحظة النتائج في جدول `job_batches` في قاعدة البيانات الذي يوضح حالة كل دفعة ونسبة الإنجاز، بالإضافة إلى ملف السجلات (Logs) الذي يظهر بدء وانتهاء كل مجموعة.

finished_at	created_at	cancelled_at	options	failed_job_ids	failed_jobs	pending_jobs	total_jobs	name	id
1778836336	1778836132	NULL	{:a:0}	[]	0	0	100	a1c8c193-6987-4290-9042-910cca58fb11	حذف نسخ تعديل
1778836540	1778836132	NULL	{:a:0}	[]	0	0	100	a1c8c194-6b62-4ac3-9fa0-015bf11a15fd	حذف نسخ تعديل
1778873353	1778873149	NULL	{:a:0}	[]	0	0	100	a1c99e37-c4cd-431d-9888-1e8e7e1b4c39	حذف نسخ تعديل
1778873557	1778873149	NULL	{:a:0}	[]	0	0	100	a1c99e37-ce5e-4895-8ab3-08a11c48811f	حذف نسخ تعديل

5. الطلب الخامس:

محاكاة توزيع الأحمال (Load Balancing Simulation)
المفهوم البرمجي: لضمان استقرار النظام عند هجوم عدد كبير من المستخدمين في وقت واحد، يجب توزيع الطلبات على أكثر من خادم (Server).

في هذا المشروع، قمنا بمحاكاة وجود خادمين (Server A و Server B) وتوزيع الطلبات بينهما لضمان عدم انهيار خادم واحد تحت الضغط.
التنفيذ البرمجي:

استخدمنا **Middleware** مخصص يسمى `LoadBalancerMiddleware`.

```
->withMiddleware(function (Middleware $middleware) {

    $middleware->validateCsrfTokens(except: [
        '/buy',
    ]);

    $middleware->alias([
        'load.balancer' => \App\Http\Middleware\LoadBalancerMiddleware::class
    ]);

})

->withExceptions(function (Exceptions $exceptions): void {
    //
})->create();
```

يعمل هذا الوسيط كشرطي مرور؛ يستقبل الطلب القادم من المتصفح، ويقوم باختيار أحد الخادمين الوهميين بناءً على خوارزمية توزيع عشوائي، ثم يوجه الطلب إليه.

الاختبار والنتائج:

عند القيام بعدة عمليات تحديث (Refresh) للمتصفح، نلاحظ في ملف storage/logs/laravel.log أن النظام يطبع رسائل مختلفة توضح الخادم الذي استلم الطلب:

- Load Balancer: Request directed to Server_A

- Load Balancer: Request directed to Server_B

```
[2026-05-15 19:25:47] local.INFO: Load Balancer: تم توجيه الطلب إلى الخادم Server_B
[2026-05-15 19:25:49] local.INFO: طلب 100 تم إطلاق دفعة معالجة متوازية لـ
[2026-05-15 19:25:49] local.INFO: طلب 100 تم إطلاق دفعة معالجة متوازية لـ
```

تم أيضاً إضافة رأس (Header) في استجابة المتصفح يسمى X-Server-Id ليوضح للمطور أي سيرفر قام بالخدمة.

الخاتمة:

بهذه الإضافات، يكون المشروع قد غطى الجوانب الأساسية للبرمجة المتوازية، بدءاً من حماية البيانات من التضارب (Locks) وصولاً إلى توزيع المهام الثقيلة (Queues) وتوزيع أحمال الشبكة (Load Balancing).